

ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΙΡΑΙΩΣ ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ

ΠΤΥΧΙΑΚΗ ΕΡΓΑΣΙΑ

ΘΕΜΑ: EstateSol: Ψηφιακή Πλατφόρμα Έξυπνης Διαχείρισης Ανακαινίσεων και Αυτοματοποιημένης Εκτίμησης Ακινήτων με τη Χρήση Τεχνολογιών Generative AI

Φοιτητής: Καραμποϊκής Νικόλαος (Α.Μ.: Π17040)

ΚΕΦΑΛΑΙΟ 1: ΕΙΣΑΓΩΓΗ

1.1 Αντικείμενο και Στόχος της Εργασίας

Αντικείμενο της παρούσας πτυχιακής εργασίας αποτελεί ο αρχιτεκτονικός σχεδιασμός, η τεχνική ανάλυση και η ολοκληρωμένη υλοποίηση της διαδικτυακής πλατφόρμας **EstateSol**. Το EstateSol συνιστά ένα πρωτοποριακό, πολυεπίπεδο πληροφοριακό σύστημα, το οποίο αναπτύχθηκε με σκοπό τον ψηφιακό μετασχηματισμό, την αυτοματοποίηση και την εισαγωγή αντικειμενικών κριτηρίων στην ελληνική αγορά ακινήτων, με έμφαση στις διαδικασίες οικιστικών ανακαινίσεων και της κτηματομεσιτικής διαμεσολάβησης.

Στην παραδοσιακή δομή της αγοράς, η επικοινωνία μεταξύ των εμπλεκόμενων μερών πάσχει από έλλειψη διαφάνειας, ασύμμετρη πληροφόρηση και κατακερματισμό των διαθέσιμων δεδομένων. Η πλατφόρμα EstateSol έρχεται να γεφυρώσει αυτό το δομικό χάσμα, λειτουργώντας ως ένα ενιαίο, ασφαλές οικοσύστημα που συνδυάζει επιχειρηματικά μοντέλα **B2C (Business-to-Consumer)** και **B2B (Business-to-Business)**. Συνδέει άμεσα τρεις βασικούς πυλώνες:

1.1.1 Σκοπός της Εφαρμογής

Σκοπός του EstateSol είναι ο ψηφιακός εκδημοκρατισμός της αγοράς ανακαίνισης. Στοχεύουμε στο να δώσουμε σε κάθε ιδιώτη ιδιοκτήτη τα εργαλεία που μέχρι πρότινος ήταν προνόμιο μόνο μεγάλων επενδυτών, καθιστώντας την ανακαίνιση μια διαφανή, προβλέψιμη και προσβάσιμη διαδικασία για όλους, ενισχύοντας παράλληλα την ποιότητα του οικιστικού αποθέματος της χώρας.

- **Ιδιώτες Ιδιοκτήτες Ακινήτων (Amateur Users):** Στους οποίους παρέχονται προηγμένα ψηφιακά εργαλεία για την αξιολόγηση της περιουσίας τους.
- **Κατασκευαστικές/Τεχνικές Εταιρείες (Construction Companies):** Οι οποίες αποκτούν πρόσβαση σε μια στοχευμένη δεξαμενή έργων (Lead Bidding Pool) χωρίς μεσάζοντες.

- **Κτηματομεσιτικά Γραφεία (Real Estate Agencies):** Τα οποία αναλαμβάνουν την εμπορική προώθηση των ακινήτων που έχουν ήδη αναβαθμιστεί, διασφαλίζοντας τη μέγιστη δυνατή απόδοση επένδυσης.

Ο κεντρικός επιστημονικός και πρακτικός στόχος της εφαρμογής είναι η παροχή μιας **αυτοματοποιημένης, αντικειμενικής μελέτης σκοπιμότητας και βιωσιμότητας (Feasibility Audit)** για κάθε καταχωρούμενο ακίνητο, σε συνδυασμό με την παραγωγή προβλέψεων για την εμπορική του αξία. Η καινοτομία αυτή επιτυγχάνεται μέσω της απευθείας ενσωμάτωσης τεχνολογιών **Παραγωγικής Τεχνητής Νοημοσύνης (Generative AI)** στο επίπεδο της επιχειρησιακής λογικής (Backend). Η πλατφόρμα δεν περιορίζεται σε μια απλή καταγραφή δεδομένων, αλλά αναλύει σε πραγματικό χρόνο τις τεχνικές παραμέτρους (τοποθεσία, τετραγωνικά μέτρα, όροφος, προϋπολογισμός) και τα οπτικά δεδομένα (φωτογραφίες) των ακινήτων.

Μέσα από τη χρήση των μοντέλων **Gemini 2.5 (Flash & Pro)** και **Imagen 3** της Google, το σύστημα παράγει αυτόματα:

1. **Οικονομικές Εκτιμήσεις:** Πρόβλεψη του πραγματικού κόστους ανακαίνισης, της μελλοντικής εμπορικής αξίας πώλησης και της δυνητικής μηνιαίας απόδοσης ενοικίου, βασισμένη σε live τάσεις της ελληνικής κτηματαγοράς.
2. **Φωτορεαλιστικές Προσομοιώσεις:** Δημιουργία μιας ψηφιακά αναβαθμισμένης εικόνας του χώρου (Image-to-Image Transformation), επιτρέποντας στον ιδιοκτήτη να οπτικοποιήσει το τελικό αποτέλεσμα πριν από την επένδυση κεφαλαίων.
3. **Δυναμική Διαμεσολάβηση (Bidding Marketplace):** Μια ολοκληρωμένη ροή δημοσίευσης σε pools, υποβολής εγγυημένων προσφορών, κλειδώματος συμβάσεων (Contract Locking) και αξιολόγησης βάσει πραγματικής συνεργασίας.

1.2 Κίνητρο (Motivation)

Το κίνητρο για τη σχεδίαση και την ανάπτυξη του EstateSol πηγάζει από την ανάλυση των χρόνιων παθογενειών που χαρακτηρίζουν τον κλάδο των κατασκευών και του Real Estate στην Ελλάδα. Η λήψη απόφασης για την ανακαίνιση ή την εμπορική εκμετάλλευση ενός ακινήτου συνοδεύεται συνήθως από υψηλό οικονομικό ρίσκο και αβεβαιότητα.

1.2.1 Το Πρόβλημα των Ιδιωτών Χρηστών

Οι ιδιώτες ιδιοκτήτες έρχονται αντιμέτωποι με την πλήρη απουσία αντικειμενικών κριτηρίων κατά το αρχικό στάδιο του σχεδιασμού. Η λήψη μιας απλής οικονομικής προσφοράς απαιτεί φυσικές αυτοψίες, χρονοβόρες διαδικασίες και υποκειμενικές κοστολογήσεις από διαφορετικούς τεχνικούς, οι οποίες συχνά αποκλίνουν σημαντικά μεταξύ τους. Επιπλέον, ο ιδιώτης αδυνατεί να γνωρίζει εκ των προτέρων αν το κεφάλαιο που σκοπεύει να επενδύσει (Budget) είναι επαρκές για τα τετραγωνικά μέτρα του ακινήτου του, ή αν η ανακαίνιση θα επιφέρει την ανάλογη αύξηση στην αξία πώλησης ή ενοικίασης (Return on Investment - ROI). Το αποτέλεσμα είναι είτε η αναστολή των έργων λόγω φόβου, είτε η υπέρβαση του προϋπολογισμού κατά την εκτέλεση.

1.2.2 Το Πρόβλημα των Τεχνικών και Μεσιτικών Εταιρειών

Από την πλευρά των επαγγελματιών, οι κατασκευαστικές εταιρείες και τα κτηματομεσιτικά γραφεία δαπανούν τεράστια ποσά και εργατοώρες σε ενέργειες μάρκετινγκ και εύρεσης πελατών (Lead Generation). Πολλές από τις επαφές αυτές αποδεικνύονται μη ποιοτικές, καθώς οι ιδιώτες μπορεί να μην διαθέτουν το απαιτούμενο κεφάλαιο ή να έχουν μη ρεαλιστικές προσδοκίες. Παράλληλα, ο υγιής ανταγωνισμός πλήττεται από την παρουσία μη πιστοποιημένων επαγγελματιών. Στην ελληνική αγορά δεν υπήρχε μια κεντρική ψηφιακή πλατφόρμα που να διενεργεί αυστηρό έλεγχο της νομιμότητας των επιχειρήσεων, διασφαλίζοντας ότι μόνο εγκεκριμένες εταιρείες με επίσημα κρατικά στοιχεία μπορούν να διεκδικήσουν έργα.

1.2.3 Η Ανάγκη για μια Ψηφιακή Λύση

Η ραγδαία εξέλιξη των μεγάλων γλωσσικών μοντέλων (LLMs) και των μοντέλων παραγωγής εικόνας προσέφερε τα τεχνικά εργαλεία για την επίλυση αυτών των προβλημάτων. Το κίνητρο της εργασίας επικεντρώθηκε στη δημιουργία ενός συστήματος που θα συγκεντρώνει όλη αυτή τη σύνθετη επιχειρησιακή λογική.

Η ανάπτυξη ενός αλγορίθμου Feasibility Audit που αξιολογεί το budget ανά τετραγωνικό μέτρο, η υλοποίηση ενός ασύγχρονου background μηχανισμού που επικοινωνεί με Cloud AI APIs, και η δημιουργία ενός clean, transactional marketplace με δυνατότητες αυτόματης ακύρωσης (voiding) προσφορών και δικαιωμάτων review βασισμένων σε πραγματική συνεργασία (hasCooperated), αποτέλεσαν την απάντηση στην ανάγκη για ένα σύγχρονο, διαφανές και πλήρως αυτοματοποιημένο περιβάλλον εργασίας.

1.2.4 Τι Πρόβλημα Λύνουμε

Το EstateSol επιλύει τρία κρίσιμα προβλήματα της ελληνικής αγοράς: Πρώτον, την ασυμμετρία πληροφόρησης (ο ιδιώτης δεν γνωρίζει το κόστος), δεύτερον, τον κατακερματισμό της αγοράς (δυσκολία εύρεσης αξιόπιστων επαγγελματιών), και τρίτον, την έλλειψη οπτικοποίησης (αδυναμία κατανόησης του τελικού αποτελέσματος). Με την πλατφόρμα μας, το πρόβλημα της αβεβαιότητας μετατρέπεται σε ένα δομημένο πλάνο μετρήσιμων αποτελεσμάτων.

1.3 Δομή της Εργασίας

Η παρούσα πτυχιακή εργασία είναι δομημένη με τέτοιο τρόπο ώστε να αποτυπώνει ολόκληρο τον κύκλο ζωής ανάπτυξης του λογισμικού (Software Development Life Cycle - SDLC), ξεκινώντας από το θεωρητικό επίπεδο και καταλήγοντας στην τεχνική αξιολόγηση και τις μελλοντικές προοπτικές.

Η εργασία διαρθρώνεται στα ακόλουθα επτά (7) εξειδικευμένα κεφάλαια:

- **ΚΕΦΑΛΑΙΟ 1: ΕΙΣΑΓΩΓΗ** Αποτελεί την εισαγωγική ενότητα της εργασίας. Καθορίζει με σαφήνεια το αντικείμενο, τους στρατηγικούς στόχους της πλατφόρμας EstateSol, το

κοινωνικό και οικονομικό κίνητρο πίσω από τη δημιουργία της, καθώς και τη συνολική δομή του τόμου.

- **ΚΕΦΑΛΑΙΟ 2: ΘΕΩΡΗΤΙΚΟ ΥΠΟΒΑΘΡΟ & ΤΕΧΝΟΛΟΓΙΕΣ** Αναλύει το επιστημονικό υπόβαθρο της αρχιτεκτονικής Client-Server και των REST APIs. Παρουσιάζει τη λεπτομερή τεχνική αιτιολόγηση για την επιλογή των εργαλείων: τη χρήση της Java 17 και του Spring Boot 3.5.9 στο backend, του reactive UI της Vue.js 3 με το Vite στο frontend, της PostgreSQL ως ACID-compliant σχεσιακής βάσης, καθώς και τη θεωρητική λειτουργία των μοντέλων Gemini 2.5 και Imagen 3 της Google στο Vertex AI.
- **ΚΕΦΑΛΑΙΟ 3: ΑΝΑΛΥΣΗ ΑΠΑΙΤΗΣΕΩΝ & ΠΡΟΔΙΑΓΡΑΦΕΣ** Επικεντρώνεται στη μηχανική απαιτήσεων (Requirements Engineering). Προσδιορίζει τους τέσσερις ρόλους των χρηστών (Actors) και καταγράφει αναλυτικά τις Λειτουργικές Απαιτήσεις (Functional Requirements) των ροών εργασίας και τις Μη Λειτουργικές Απαιτήσεις (Non-Functional Requirements) που αφορούν την ασφάλεια, την απόδοση και την ανθεκτικότητα.
- **ΚΕΦΑΛΑΙΟ 4: ΑΡΧΙΤΕΚΤΟΝΙΚΗ ΚΑΙ ΣΧΕΔΙΑΣΜΟΣ ΣΥΣΤΗΜΑΤΟΣ** Παρουσιάζει τον δομικό σχεδιασμό της πλατφόρμας. Περιλαμβάνει την πλήρη ανάλυση του Directory Structure του πηγαίου κώδικα (Backend και Frontend) και αναλύει το Σχεσιακό

Διάγραμμα Οντοτήτων-Συσχετίσεων (ER Diagram) της βάσης δεδομένων, επεξηγώντας τα foreign keys και τους περιορισμούς ακεραιότητας.

- **ΚΕΦΑΛΑΙΟ 5: ΥΛΟΠΟΙΗΣΗ ΣΥΣΤΗΜΑΤΟΣ** Το εκτενέστερο κεφάλαιο της εργασίας, το οποίο εμβαθύνει στον κώδικα. Αναλύει τη λειτουργία όλων των REST Controllers, την επιχειρησιακή λογική των Core Services (με έμφαση στην ασύγχρονη επεξεργασία `@Async` και τον αλγόριθμο Feasibility), τις ρυθμίσεις ασφαλείας του Spring Security (Sessions, Remember-Me), και την reactive αρχιτεκτονική της Vue.js 3 με το custom σύστημα διαχείρισης state.
- **ΚΕΦΑΛΑΙΟ 6: ΕΛΕΓΧΟΣ ΚΑΙ ΑΞΙΟΛΟΓΗΣΗ** Παρουσιάζει τις μεθοδολογίες ελέγχου που εφαρμόστηκαν. Αναλύει τα test cases για τις ροές των Bids (Voiding mechanism), τον έλεγχο προσβάσεων βάσει ρόλων (RBAC), και την αξιολόγηση της συμπεριφοράς του συστήματος υπό συνθήκες Rate Limiting με την επιτυχή ενεργοποίηση του Fallback Pricing Service.
- **ΚΕΦΑΛΑΙΟ 7: ΣΥΜΠΕΡΑΣΜΑΤΑ & ΜΕΛΛΟΝΤΙΚΕΣ ΕΠΕΚΤΑΣΕΙΣ** Συνοψίζει τα τελικά συμπεράσματα και τα οφέλη από την υλοποίηση του EstateSol. Καταγράφει τις τεχνικές προκλήσεις και τα προβλήματα που επιλύθηκαν κατά την ανάπτυξη, και παραθέτει συγκεκριμένες προτάσεις για τη μελλοντική εξέλιξη της πλατφόρμας στην παραγωγική αγορά.

1.4 Κοινωνικό Αποτύπωμα

Η πλατφόρμα **EstateSol** προσφέρει σημαντικό κοινωνικό όφελος στην ελληνική κοινωνία, συμβάλλοντας στην αντιμετώπιση του στεγαστικού προβλήματος. Μέσω της ανακαίνισης και ενεργειακής αναβάθμισης του παλαιού κτιριακού αποθέματος της χώρας, η πλατφόρμα αυξάνει τον αριθμό των ποιοτικών και διαθέσιμων ενοικιαζόμενων διαμερισμάτων. Η αύξηση της προσφοράς ανακαινισμένων ακινήτων λειτουργεί εξισορροπητικά στην αγορά, ασκώντας

πιέσεις για τη μείωση της μέσης τιμής των ακινήτων και καθιστώντας τη στέγαση πιο προσιτή για τους πολίτες.

1.5 Έρευνα Αγοράς και Ανταγωνιστικό Πλεονέκτημα

1.5.1 Οι Ελληνικές Πλατφόρμες (Τοπικός Ανταγωνισμός)

Στην εγχώρια αγορά, πλατφόρμες όπως το InteriorAI.gr ή το AkinitaAI.gr επικεντρώνονται σε μεμονωμένες υπηρεσίες, ενώ παραδοσιακά sites τύπου "Douleutaras" λειτουργούν κυρίως ως κατάλογοι τεχνιτών. Το EstateSol διαφοροποιείται υιοθετώντας μια ολιστική προσέγγιση, καθώς είναι το μοναδικό σύστημα που συνδυάζει την παραγωγή AI εικόνας, την αυτόματη κοστολόγηση και ένα κλειστό transactional marketplace.

1.5.2 Ξένα AI Εργαλεία Κοστολόγησης (SaaS Tools)

Σε διεθνές επίπεδο, εργαλεία όπως το GetEstimate.ai και το Renovation Calculator AI λειτουργούν ως απλοί υπολογιστές (calculators) χωρίς σύνδεση με την πραγματική αγορά. Αντίθετα, το EstateSol λειτουργεί ως ολοκληρωμένο Marketplace, γεφυρώνοντας τη θεωρητική εκτίμηση με την πραγματική ανάθεση έργου σε πιστοποιημένους επαγγελματίες.

1.5.3 AI Εφαρμογές Σχεδιασμού (Μόνο Εικόνα)

Διεθνείς εφαρμογές όπως το DecAI.ai και το Interior AI περιορίζονται στην οπτική αναβάθμιση χώρων χωρίς να λαμβάνουν υπόψη το οικονομικό σκέλος. Το EstateSol υπερέχει συνδέοντας την AI εικόνα (μέσω του Imagen 3) απευθείας με το πραγματικό budget του ιδιοκτήτη και τις τρέχουσες τιμές της ελληνικής αγοράς.

1.5.4 Στρατηγικό Πλεονέκτημα

Το EstateSol αποτελεί το μοναδικό οικοσύστημα που συνδυάζει τρεις πυλώνες: την ανακαίνιση βάσει ρεαλιστικού budget, τον αυτόματο υπολογισμό εμπορικής αξίας και την εγγυημένη σύναψη συμφωνίας με πιστοποιημένες εταιρείες, προσφέροντας μια αδιάσπαστη αλυσίδα αξίας από την ιδέα μέχρι την υλοποίηση.

ΚΕΦΑΛΑΙΟ 2: ΘΕΩΡΗΤΙΚΟ ΥΠΟΒΑΘΡΟ & ΤΕΧΝΟΛΟΓΙΕΣ

2.1 Αρχιτεκτονική Client-Server και RESTful APIs

Η σύγχρονη μηχανική λογισμικού (Software Engineering) για τη δημιουργία διαδικτυακών συστημάτων enterprise επιπέδου επιβάλλει τον πλήρη διαχωρισμό του επιπέδου παρουσίασης (Frontend) από το επίπεδο της επιχειρησιακής λογικής και των δεδομένων (Backend). Η προσέγγιση αυτή, γνωστή ως **Decoupled Architecture** (Αποσυζευγμένη Αρχιτεκτονική), προσφέρει μέγιστη ευελιξία, καθώς επιτρέπει στα δύο υποσυστήματα να αναπτύσσονται, να συντηρούνται και να επεκτείνονται αυτόνομα, χρησιμοποιώντας διαφορετικά τεχνολογικά stacks.

Η επικοινωνία μεταξύ της Client-Side εφαρμογής και του Server-Side εξυπηρετητή βασίζεται αποκλειστικά στις αρχές της αρχιτεκτονικής προσέγγισης **REST (Representational State Transfer)** μέσω του πρωτοκόλλου HTTP/HTTPS. Τα βασικά χαρακτηριστικά της REST αρχιτεκτονικής που ενσωματώθηκαν στο EstateSol περιλαμβάνουν:

- **Statelessness (Απουσία Κατάστασης):** Κάθε HTTP request που αποστέλλεται από τον client προς το backend API είναι απόλυτα αυτόνομο. Δεν βασίζεται σε αποθηκευμένο ιστορικό στον server, αλλά περιέχει όλη την απαραίτητη πληροφορία (δεδομένα, session context) για την εξυπηρέτησή του. Αυτό επιτρέπει την οριζόντια κλιμάκωση (horizontal scaling) του backend χωρίς την ανάγκη συγχρονισμού πολύπλοκων distributed sessions.
- **Uniform Interface (Ενιαία Διεπαφή):** Η διαχείριση των πόρων (Resources), όπως τα ακίνητα (**Apartment**) ή οι προσφορές (**LeadBid**), γίνεται με τυποποιημένο τρόπο μέσω των HTTP ρημάτων:
 - **GET:** Για την ανάκτηση δεδομένων.
 - **POST:** Για τη δημιουργία νέων εγγγραφών ή την εκτέλεση κρίσιμων transactional ενεργειών.
- **Ανταλλαγή Δομημένων Δεδομένων:** Η μεταφορά των πληροφοριών πραγματοποιείται αποκλειστικά σε μορφή **JSON (JavaScript Object Notation)**, η οποία αποτελεί το standard της βιομηχανίας λόγω του ελαφρού χαρακτήρα της και της εγγενούς

συμβατότητάς της τόσο με την Java (μέσω του Jackson Object Mapper) όσο και με τη JavaScript της Vue.js.

2.2 Τεχνολογίες Backend (Server-Side)

Το Backend API του EstateSol σχεδιάστηκε με γνώμονα τη σταθερότητα, την ασφάλεια, την ανθεκτικότητα σε σφάλματα και την ικανότητα διαχείρισης ασύγχρονων ροών εργασίας. Για τον λόγο αυτό, επιλέχθηκε το οικοσύστημα της γλώσσας Java και το framework Spring Boot.

2.2.1 Java 17 (Long-Term Support)

Η επιλογή της **Java 17** ως της βασικής γλώσσας προγραμματισμού του backend έγινε με βάση τον Long-Term Support (LTS) χαρακτήρα της, ο οποίος διασφαλίζει μακροχρόνια σταθερότητα, ενημερώσεις ασφαλείας και βελτιστοποιημένη απόδοση του Java Virtual Machine (JVM). Η Java 17 εισάγει κρίσιμα χαρακτηριστικά που αξιοποιήθηκαν για τη διασφάλιση της ποιότητας του κώδικα (Clean Code):

- **Records:** Χρησιμοποιήθηκαν εκτενώς για τη δημιουργία των Data Transfer Objects (DTOs), όπως το `RegisterRequest` και το `ApartmentCreateDto`. Τα Records παράγουν αυτόματα immutable (αμετάβλητα) αντικείμενα δεδομένων, εκμηδενίζοντας τον boilerplate κώδικα (getters, setters, equals, hashCode, toString) και θωρακίζοντας τη μεταφορά δεδομένων μεταξύ των επιπέδων.
- **Pattern Matching για switch & Sealed Classes:** Επέτρεψαν τη δημιουργία πιο καθαρών και ασφαλών δομών ελέγχου κατά τη διαχείριση των Enums των ρόλων των χρηστών (`UserRole`) και των καταστάσεων των έργων (`ProjectStatus`), ελαχιστοποιώντας την πιθανότητα runtime σφαλμάτων.
- **Βελτιωμένη Διαχείριση Μνήμης:** Οι αναβαθμισμένοι αλγόριθμοι Garbage Collection (όπως ο ZGC) προσφέρουν εξαιρετικά χαμηλούς χρόνους παύσης (low-latency), στοιχείο απαραίτητο για τη διαχείριση των multipart image uploads και των ασύγχρονων AI διεργασιών της πλατφόρμας.

2.2.2 Spring Boot 3.5.9

Το framework **Spring Boot 3.x** αποτελεί τη ναυαρχίδα της enterprise ανάπτυξης σε Java, εφαρμόζοντας τη φιλοσοφία του *Convention over Configuration* (Σύμβαση έναντι Παραμετροποίησης). Μέσω των Starter Dependencies, αυτοματοποιεί τη ρύθμιση του ενσωματωμένου εξυπηρετητή Apache Tomcat και των απαραίτητων βιβλιοθηκών. Στο EstateSol χρησιμοποιήθηκαν οι εξής εξειδικευμένες συνιστώσες:

- **Spring Boot Starter Web:** Υπεύθυνο για την υλοποίηση της αρχιτεκτονικής MVC (Model-View-Controller) στο API επίπεδο. Αναλαμβάνει τη δρομολόγηση των HTTP URLs στους κατάλληλους Controllers και τη μετατροπή των Java αντικειμένων σε JSON και αντίστροφα.
- **Spring Boot Starter Data JPA & Hibernate:** Το Java Persistence API (JPA) σε συνδυασμό με το Hibernate ως Object-Relational Mapper (ORM), γεφυρώνει το χάσμα

μεταξύ του αντικειμενοστραφούς προγραμματισμού και της σχεσιακής βάσης δεδομένων. Οι πίνακες της PostgreSQL χαρτογραφούνται σε Java κλάσεις (`@Entity`), και η διαχείριση των ερωτημάτων γίνεται μέσω των έτοιμων interfaces του

`JpaRepository`, εξαλείφοντας την ανάγκη για χειροκίνητη συγγραφή ευπαθών SQL queries.

- **Spring Boot Starter Security:** Το κεντρικό υποσύστημα ασφαλείας. Παρέχει προηγμένους μηχανισμούς για την ταυτοποίηση (Authentication) και την εξουσιοδότηση (Authorization), προστατεύοντας τα endpoints βάσει των δικαιωμάτων του εκάστοτε ρόλου, ενώ διαχειρίζεται τα session cookies και τα remember-me persistent tokens.
- **Spring Boot Starter WebFlux (WebClient):** Σε αντίθεση με το παλαιότερο, μπλοκαριστικό `RestTemplate`, το `WebClient` χρησιμοποιεί μια non-blocking, reactive αρχιτεκτονική. Αυτό είναι κρίσιμο για το EstateSol, καθώς επιτρέπει την εκτέλεση HTTP κλήσεων προς τα APIs του Google Vertex AI χωρίς να δεσμεύει και να παγώνει τα threads του backend, διασφαλίζοντας υψηλή απόδοση υπό μεγάλο φόρτο.
- **Spring Boot Starter Actuator:** Παρέχει live endpoints παρακολούθησης της υγείας (health checks) και των metrics του συστήματος, προσφέροντας στον Admin εικόνα για την κατάσταση του server σε περιβάλλον παραγωγής.
- **Project Lombok:** Μια βιβλιοθήκη επεξεργασίας annotations που χρησιμοποιείται για τη δυναμική παραγωγή boilerplate κώδικα (όπως `@Getter`, `@Setter`, `@NoArgsConstructor`, `@AllArgsConstructor` και `@Builder` στις οντότητες), διατηρώντας τον πηγαίο κώδικα εξαιρετικά λιτό και ευανάγνωστο.

2.3 Τεχνολογίες Frontend (Client-Side)

Το επίπεδο παρουσίασης σχεδιάστηκε με στόχο τη δημιουργία μιας εξαιρετικά γρήγορης, αποκρινόμενης (responsive) και fluid διεπαφής χρήστη, η οποία θα προσφέρει την αίσθηση μιας native desktop εφαρμογής.

2.3.1 Vue.js 3 (Composition API)

Το **Vue.js 3** επιλέχθηκε ως το κεντρικό προοδευτικό framework (Progressive Framework) για τη δόμηση του Single Page Application (SPA). Η επιλογή του βασίστηκε στην ανώτερη απόδοση του αναβαθμισμένου Virtual DOM του και στην υιοθέτηση του **Composition API** (μέσω του `<script setup>`). Το Composition API επιτρέπει:

- **Λειτουργική Οργάνωση (Feature Grouping):** Η επιχειρησιακή λογική των components οργανώνεται με βάση τη λειτουργικότητά τους και όχι με βάση αυστηρά options (data, methods, computed), διευκολύνοντας τη συντήρηση μεγάλων αρχείων (όπως το `AdminDashboardView.vue` ή το `BusinessProfileView.vue`).
- **Επαναχρησιμοποίηση Κώδικα (Composables):** Επιτρέπει την εξαγωγή κοινής reactive λογικής σε αυτόνομα αρχεία JavaScript, βελτιώνοντας την αρχιτεκτονική του frontend.
- **Απόλυτη Αντιδραστικότητα (Fine-Grained Reactivity):** Μέσω των reactive primitives `ref()` και `reactive()`, η Vue εντοπίζει με ακρίβεια ποιο σημείο του DOM πρέπει να

ανανεωθεί όταν αλλάζουν τα δεδομένα, προσφέροντας ακαριαία οπτική ενημέρωση (π.χ. κατά το live φιλτράρισμα των ακινήτων στο [ConstructionPool.vue](#)).

2.3.2 Vite

Το **Vite** χρησιμοποιήθηκε ως το build tool και ο dev server επόμενης γενιάς, αντικαθιστώντας το παραδοσιακό Webpack. Το Vite εκμεταλλεύεται τα native ES Modules (ESM) του browser, πράγμα που σημαίνει ότι δεν χρειάζεται να κάνει bundle ολόκληρο τον κώδικα κατά την ανάπτυξη. Αυτό προσφέρει:

- Σχεδόν ακαριαία εκκίνηση του τοπικού server.
- Ακαριαίο Hot Module Replacement (HMR), όπου οι αλλαγές στον κώδικα αντανakλώνται στον browser σε ελάχιστα χιλιοστά του δευτερολέπτου.
- Εξαιρετικά βελτιστοποιημένη διαδικασία building για την παραγωγή (production bundle) μέσω του Rollup, ελαχιστοποιώντας το μέγεθος των αρχείων που κατεβάζει ο τελικός χρήστης.

2.3.3 Διαχείριση Κατάστασης (State Management) & Πλοήγηση

- **Vue Reactive Session Object ([auth.js](#)):** Για τη διαχείριση της παγκόσμιας κατάστασης (Global State), το EstateSol χρησιμοποιεί ένα custom **reactive session object** στο αρχείο [src/stores/auth.js](#). Αυτή η ελαφριά προσέγγιση αντικαθιστά τα βαριά stores, επιτρέποντας την άμεση παρακολούθηση της μεταβλητής [isAuthenticated](#), των δεδομένων του [user](#) και της αυτόματης κανονικοποίησης (normalization) των ρόλων από τη μορφή της Spring Security (π.χ. [ROLE_ADMIN](#)) σε καθαρά strings αναγνώσιμα από το UI (π.χ. [ADMIN](#)).
- **Vue Router 4:** Ο επίσημος δρομολογητής για το Vue.js. Αναλαμβάνει τη διαχείριση των URLs, τη φορτωτική βελτιστοποίηση μέσω lazy-loading (ασύγχρονη φόρτωση σελίδων, όπως το [BusinessProfileView.vue](#) μόνο όταν ζητηθεί) και την υλοποίηση των **Navigation Guards**, οι οποίοι ελέγχουν τα δικαιώματα πρόσβασης πριν από τη φόρτωση κάθε σελίδας.

2.4 Σχεσιακές Βάσεις Δεδομένων: PostgreSQL

Για την ασφαλή και μόνιμη αποθήκευση των δεδομένων (Persistence Layer) επιλέχθηκε η **PostgreSQL**. Πρόκειται για ένα enterprise-level, open-source αντικειμενοστραφές σχεσιακό σύστημα διαχείρισης βάσεων δεδομένων (RDBMS). Τα κριτήρια επιλογής της PostgreSQL περιλαμβάνουν:

- **Απόλυτη Συμμόρφωση ACID:** Εγγυάται ότι όλες οι κρίσιμες συναλλαγές του marketplace (όπως η αποδοχή ενός bid, η οποία πρέπει ταυτόχρονα να κλειδώσει το project και να απορρίψει όλες τις άλλες προσφορές) εκτελούνται ως μια αδιαίρετη

μονάδα εργασίας (Atomicity), διασφαλίζοντας την ακεραιότητα των δεδομένων (Consistency).

- **Προηγμένη Διαχείριση Ταυτόχρονων Χρηστών:** Μέσω του μηχανισμού MVCC (Multi-Version Concurrency Control), η PostgreSQL επιτρέπει την ταυτόχρονη ανάγνωση και εγγραφή δεδομένων χωρίς να μπλοκάρει τους χρήστες, στοιχείο απαραίτητο για ένα ζωντανό bidding pool.
- **Άριστη Συνεργασία με το Hibernate:** Υποστηρίζει πλήρως τη δυναμική ενημέρωση του schema (`ddl-auto=update`), τους αυτόματους περιορισμούς ξένων κλειδίων (Foreign Keys) με cascade διαγραφές, και τη φιλοξενία των πινάκων persistent logs του Spring Security.

2.5 Τεχνολογίες Τεχνητής Νοημοσύνης (Generative AI & Vertex AI)

Η επιχειρηματική διαφοροποίηση και η καινοτομία του EstateSol βασίζονται στην ενσωμάτωση Generative AI μοντέλων της Google μέσω της enterprise πλατφόρμας **Vertex AI**. Η επικοινωνία υλοποιείται με ασφαλή έλεγχο ταυτότητας (Google Application Default Credentials - ADC) μέσω των επίσημων Java SDKs.

2.5.1 Μοντέλο Gemini 2.5 (Flash & Pro)

Το **Gemini 2.5** είναι ένα υπερσύγχρονο, πολυτροπικό (multimodal) μεγάλο γλωσσικό μοντέλο. Στο EstateSol αξιοποιούνται δύο εξειδικευμένες εκδόσεις του:

- **Gemini-2.5-flash:** Επιλέχθηκε για το επίπεδο των οικονομικών εκτιμήσεων λόγω της εξαιρετικής ταχύτητας απόκρισης και του χαμηλού κόστους λειτουργίας. Το μοντέλο δέχεται ως είσοδο ένα δομημένο prompt που περιλαμβάνει την περιοχή, τα τετραγωνικά, τον όροφο και την κατηγορία βιωσιμότητας του ακινήτου. Αξιοποιώντας προηγμένες τεχνικές prompting (Few-Shot Prompting, Structured Output Enforcement), το μοντέλο παρακάμπτει την παραγωγή απλού κειμένου και επιστρέφει ένα αυστηρό αντικείμενο **JSON**. Αυτό το JSON περιέχει τις προβλέψεις για το μέσο ενοίκιο, την αξία πώλησης και το εκτιμώμενο κόστος ανακαίνισης, βασισμένο σε live δεδομένα της ελληνικής κτηματαγοράς, επιτρέποντας στο backend να τα αποθηκεύσει απευθείας στα αντίστοιχα double fields της βάσης δεδομένων.
- **Gemini-2.5-pro:** Χρησιμοποιείται στη ροή ανάλυσης εικόνας. Λόγω των ανώτερων συλλογιστικών ικανοτήτων του (advanced reasoning), αναλύει την αρχική φωτογραφία του παλαιού δωματίου που ανέβασε ο χρήστης και παράγει ένα εξειδικευμένο, λεπτομερές Prompt σχεδιασμού εσωτερικών χώρων, προσαρμοσμένο στην κατηγορία του διαθέσιμου budget (π.χ. Cosmetic ή Premium).

2.5.2 Μοντέλο Imagen 3

Το **imagen-3.0-capability-001** αποτελεί το κορυφαίο μοντέλο παραγωγής και τροποποίησης εικόνων της Google. Στο EstateSol, το Imagen 3 αξιοποιείται για τη διαδικασία του **Image-to-Image Editing**.

Το backend στέλνει στο API την αρχική φωτογραφία του ακινήτου σε μορφή raw base6 bytes (`REFERENCE_TYPE_RAW`), μαζί με το Prompt σχεδιασμού που παρήγαγε το Gemini 2.5 Pro. Το μοντέλο Image 3 επεξεργάζεται τα δεδομένα και παράγει μια νέα, φωτορεαλιστική εικόνα που προσομοιώνει το δωμάτιο μετά την ανακαίνιση. Η χρήση των raw αναφορών είναι δομικής σημασίας, καθώς διασφαλίζει ότι η νέα εικόνα διατηρεί απόλυτα σταθερή τη γεωμετρία του χώρου, τη διαρρύθμιση των τοίχων, τις θέσεις των παραθύρων και τις γωνίες λήψης της κάμερας, αλλάζοντας μόνο τα υλικά, τα χρώματα, τους φωτισμούς και τη διακόσμηση βάσει του budget του ιδιοκτήτη.

ΚΕΦΑΛΑΙΟ 3: ΑΝΑΛΥΣΗ ΑΠΑΙΤΗΣΕΩΝ & ΠΡΟΔΙΑΓΡΑΦΕΣ

3.1 Ρόλοι Χρηστών (Actors)

Η αρχιτεκτονική του EstateSol βασίζεται σε ένα δυναμικό σύστημα διαχείρισης χρηστών και δικαιωμάτων πρόσβασης, το οποίο μοντελοποιείται μέσω του Enum `UserRole`. Το σύστημα αναγνωρίζει τέσσερις (4) διακριτούς ρόλους, καθένας από τους οποίους ενεργοποιεί συγκεκριμένες επιχειρησιακές ροές (workflows) και έχει πρόσβαση σε εξειδικευμένα endpoints του REST API:

- Ιδιώτης Χρήστης / Ιδιοκτήτης (`AMATEUR_USER`):** Ο ρόλος αυτός αντιπροσωπεύει τον τελικό καταναλωτή της πλατφόρμας. Έχει το δικαίωμα να εισάγει τα ακίνητά του, να μεταφορτώνει φωτογραφίες και να ορίζει το επιθυμητό budget. Είναι ο αποκλειστικός αποδέκτης των αυτοματοποιημένων AI εκτιμήσεων και των οπτικών προσομοιώσεων. Επιπλέον, έχει τη δυνατότητα να δημοσιεύει το ακίνητό του στα δημόσια pools, να εξετάζει τις εισερχόμενες προσφορές των εταιρειών, να τις αποδέχεται ή να τις απορρίπτει και να υποβάλλει reviews για τις εταιρείες με τις οποίες έχει συνεργαστεί.
- Κατασκευαστική Εταιρεία (`CONSTRUCTION_COMPANY`):** Πρόκειται για εταιρικό λογαριασμό B2B. Κατά την εγγραφή, η εταιρεία υποχρεούται να καταθέσει τα επίσημα ταυτοποιητικά της στοιχεία (ΑΦΜ και αριθμό ΓΕΜΗ). Μέχρι να εγκριθεί από τον διαχειριστή, ο λογαριασμός παραμένει σε κατάσταση αναμονής (`isVerified = false`). Μόλις επαληθευτεί, αποκτά πρόσβαση στο *Construction Pool*, όπου μπορεί να φιλτράρει ενεργά έργα ανακαίνισης, να βλέπει λεπτομέρειες (όπως Google Maps Pins) και να καταθέτει οικονομικές προσφορές και τεχνικά υπομνήματα (`LeadBid`). Επίσης, διαθέτει εταιρικό wall για τη δημοσίευση posts.
- Κτηματομεσιτικό Γραφείο (`REAL_ESTATE_AGENCY`):** Αντίστοιχος εταιρικός B2B ρόλος, ο οποίος υπόκειται στην ίδια αυστηρή διαδικασία ελέγχου εγγράφων (ΑΦΜ/ΓΕΜΗ). Η διαφοροποίησή του έγκειται στο επιχειρησιακό του αντικείμενο: αντί για το κατασκευαστικό pool, έχει πρόσβαση στο *Real Estate Pool*. Εκεί εμφανίζονται ακίνητα που είτε έχουν ολοκληρώσει το στάδιο της ανακαίνισης (`COMPLETED`), είτε έχουν δημοσιευθεί απευθείας για μεσιτική ανάληψη. Στόχος του είναι η υποβολή προσφορών για την ανάληψη της προώθησης (πώληση ή ενοικίαση) του ακινήτου.

4. **Διαχειριστής (ADMIN):** Ο ανώτατος ρόλος του συστήματος, ο οποίος προστατεύεται καθολικά από το Spring Security. Ο Admin δεν συμμετέχει στο marketplace ως αγοραστής ή πωλητής, αλλά εποπτεύει τη σωστή λειτουργία του. Έχει πρόσβαση σε ειδικό panel διαχείρισης χρηστών (`UserManagement.vue`), όπου ελέγχει και εγκρίνει τις εταιρείες (`verify`), διορθώνει χειροκίνητα λάθος στοιχεία εγγράφων (`override`), διαγράφει κακόβουλους λογαριασμούς και έχει το δικαίωμα να αλλάξει καθολικά το status οποιουδήποτε project για λόγους συντήρησης της βάσης.

3.2 Λειτουργικές Απαιτήσεις (Functional Requirements)

Οι λειτουργικές απαιτήσεις περιγράφουν τις θεμελιώδεις συμπεριφορές και τις υπηρεσίες που εκτελεί το σύστημα. Για το EstateSol, οι απαιτήσεις αυτές οργανώνονται σε πέντε κρίσιμα υποσυστήματα:

3.2.1 Υποσύστημα Ταυτοποίησης και Εγγραφής

- Το σύστημα πρέπει να επιτρέπει την εγγραφή χρηστών διαχωρίζοντας τους ρόλους μέσω ειδικών tabs στο UI.
- Αν ο χρήστης επιλέξει εταιρικό ρόλο, το σύστημα πρέπει να καθιστά υποχρεωτική την εισαγωγή 9-ψήφιου ΑΦΜ, αριθμού ΓΕΜΗ και γεωγραφικής ζώνης δραστηριοποίησης (Region).
- Το σύστημα πρέπει να απαγορεύει τη χειροκίνητη εγγραφή χρηστών με ρόλο `ADMIN` μέσω του δημόσιου endpoint.

3.2.2 Υποσύστημα Διαχείρισης Ακινήτων και AI Επεξεργασίας

- Ο ιδιοκτήτης πρέπει να μπορεί να καταχωρεί ένα ακίνητο εισάγοντας: περιοχή, τ.μ., όροφο, σύνδεσμο Google Maps Pin, budget και μια φωτογραφία.
- Κατά την αποθήκευση, το σύστημα πρέπει να θέτει το ακίνητο σε κατάσταση `DRAFT` με την ένδειξη `aiProcessed = false` και να πυροδοτεί άμεσα το ασύγχρονο background thread επεξεργασίας.
- Το σύστημα πρέπει να υπολογίζει τη βιωσιμότητα βάσει του αλγορίθμου feasibility. Αν το budget είναι ανεπαρκές ($\$ < 200€ \backslash \text{t}\{\text{t.}\mu.\}\$$), πρέπει να διακόπτει την AI ροή, να ενημερώνει το πεδίο `estimatedCost` με την ένδειξη `UNFEASIBLE` και να μην προχωρά σε κλήσεις των Vertex AI APIs για την αποφυγή άσκοπης σπατάλης πόρων.
- Αν το έργο είναι βιώσιμο, το σύστημα πρέπει να ανακτά τις τιμές (rent, sale, renovation cost) από το Gemini 2.5, να παράγει την ανακαινισμένη εικόνα μέσω του Imagen 3, να την αποθηκεύει τοπικά και να μετατρέπει το status του ακινήτου σε `VISUALIZED` με `aiProcessed = true`.

3.2.3 Υποσύστημα Marketplace (Pools & Bids)

- Το σύστημα πρέπει να επιτρέπει στον ιδιοκτήτη να δημοσιεύει το ακίνητό του στα δημόσια pools (`POST /api/pool/publish/{id}`). Αν το ακίνητο είναι νέο, μεταφέρεται στο `POOL_CONSTRUCTION`. Αν έχει ολοκληρωθεί η ανακαίνιση, μεταφέρεται στο `POOL_REAL_ESTATE`.
- Το σύστημα πρέπει να επιτρέπει μόνο σε επαληθευμένες εταιρείες (`isVerified = true`) να υποβάλουν προσφορά (`LeadBid`), εισάγοντας το δικό τους εκτιμώμενο κόστος και τεχνικές σημειώσεις.
- Όταν ο ιδιοκτήτης επιλέξει «Αποδοχή Προσφοράς» σε ένα bid, το σύστημα πρέπει να εκτελεί μια αδιαίρετη συναλλαγή (ACID Transaction): να αλλάζει το status του ακινήτου

σε **CONTRACT_LOCKED**, να θέτει το συγκεκριμένο bid ως **ACCEPTED** και να μετατρέπει αυτόματα όλα τα υπόλοιπα ανταγωνιστικά bids του ακινήτου σε **REJECTED**.

- Αν ο ιδιοκτήτης επιλέξει να κάνει update ένα ακίνητο που βρίσκεται στο pool, το σύστημα πρέπει να διαγράφει αυτόματα (**voiding**) όλες τις εκκρεμείς προσφορές, καθώς τα νέα τεχνικά δεδομένα αλλάζουν το οικονομικό αντικείμενο του έργου.

3.2.4 Υποσύστημα Εταιρικών Προφίλ, Posts και Reviews

- Το σύστημα πρέπει να εμφανίζει έναν δημόσιο κατάλογο εταιρειών, υπολογίζοντας δυναμικά τον μέσο όρο των κριτικών (1-10) και το πλήθος των posts.
- Μια εταιρεία πρέπει να μπορεί να δημοσιεύει ενημερώσεις και φωτογραφίες έργων στο wall της, υπό την προϋπόθεση ότι ο λογαριασμός της είναι επαληθευμένος.
- Το σύστημα πρέπει να επιτρέπει σε έναν ιδιώτη να υποβάλει κριτική και βαθμολογία για μια εταιρεία **MONO** αν η μεταβλητή **hasCooperated** είναι αληθής (**true**), δηλαδή αν υπάρχει στη βάση δεδομένων εγκεκριμένο συμβόλαιο (**ACCEPTED bid**) μεταξύ του συγκεκριμένου ιδιώτη και της συγκεκριμένης εταιρείας.

3.3 Μη Λειτουργικές Απαιτήσεις (Non-Functional Requirements)

Οι μη λειτουργικές απαιτήσεις καθορίζουν τα ποιοτικά χαρακτηριστικά, τα τεχνικά κριτήρια και τους περιορισμούς υπό τους οποίους πρέπει να λειτουργεί το σύστημα.

3.3.1 Ασφάλεια και Θωράκιση Δεδομένων

- **Κρυπτογράφηση:** Όλοι οι κωδικοί πρόσβασης των χρηστών πρέπει να αποθηκεύονται στη βάση δεδομένων PostgreSQL αποκλειστικά σε κρυπτογραφημένη μορφή, κάνοντας χρήση του αλγορίθμου ισχυρού κατακερματισμού **BCrypt** με salt.
- **Προστασία Endpoints:** Η πρόσβαση στα endpoints πρέπει να ελέγχεται αυστηρά σε επίπεδο φίλτρων (URL-based security) και σε επίπεδο μεθόδων κώδικα (Method-level security). Οι Admins έχουν αποκλειστική πρόσβαση στο path **/api/admin/**. • **Διαχείριση Συνεδριών:** Επιβάλλεται ο έλεγχος ταυτόχρονων συνεδριών (**maximumSessions(1)**). Αν ένας χρήστης συνδεθεί από νέα συσκευή, η παλαιά συνεδρία πρέπει να ακυρώνεται άμεσα για την αποτροπή κακόβουλης χρήσης.

3.3.2 Απόδοση και Διαχειριστική Απόκριση

- **Ασύγχρονη Λειτουργία:** Οι κλήσεις προς τα APIs του Vertex AI, οι οποίες μπορεί να διαρκέσουν από 3 έως 8 δευτερόλεπτα (ειδικά για την παραγωγή εικόνας από το Imagen 3), απαγορεύεται να μπλοκάρουν το main thread της Spring Boot ή να παγώνουν το UI του client. Η δημιουργία της εγγραφής στη βάση πρέπει να επιστρέφει HTTP 200 OK σε λιγότερο από 200ms, και η AI επεξεργασία να εκτελείται αυτόνομα στο παρασκήνιο.
- **Μέγεθος Bundle:** Το frontend UI πρέπει να είναι εξαιρετικά ελαφρύ. Η χρήση του Vite διασφαλίζει ότι το production bundle είναι ελαχιστοποιημένο, επιτρέποντας τη γρήγορη φόρτωση της σελίδας ακόμα και σε συνδέσεις δικτύου χαμηλής ταχύτητας.

3.3.3 Διαθεσιμότητα και Ανθεκτικότητα σε Σφάλματα (Fault Tolerance)

- **Μηχανισμός Fallback:** Το σύστημα πρέπει να είναι ανθεκτικό σε περιπτώσεις Rate Limiting (HTTP Status 429 Too Many Requests) ή προσωρινής διακοπής των υπηρεσιών της Google Cloud. Αν το **GeminiClient** αποτύχει, το σύστημα πρέπει να εκτελεί αυτόματη εκτροπή (failover) στο **FallbackPricingService**, αντλώντας στατικές τιμές από τοπικούς πίνακες, ώστε ο χρήστης να λαμβάνει πάντα μια αξιόπιστη εκτίμηση χωρίς να κρασάρει η εφαρμογή.

3.3.4 Επεκτασιμότητα και Φορητότητα (Scalability & Portability)

- **Deployment-Ready Audit:** Η εφαρμογή δεν πρέπει να περιέχει hardcoded ρυθμίσεις (όπως passwords βάσης, API keys ή IP addresses). Όλες οι κρίσιμες παράμετροι στο `application.properties` πρέπει να είναι συνδεδεμένες με Περιβαλλοντικές Μεταβλητές (Environment Variables). Αυτό διασφαλίζει ότι το EstateSol μπορεί να γίνει build, να τρέξει τοπικά ή να μεταφερθεί άμεσα σε περιβάλλον Cloud (GCP, AWS) ή Docker Containers χωρίς καμία αλλαγή στον πηγαίο κώδικα.

ΚΕΦΑΛΑΙΟ 4: ΑΡΧΙΤΕΚΤΟΝΙΚΗ ΚΑΙ ΣΧΕΔΙΑΣΜΟΣ ΣΥΣΤΗΜΑΤΟΣ

4.1 Γενική Αρχιτεκτονική Προσέγγιση

Η πλατφόρμα **EstateSol** βασίζεται σε μια σύγχρονη, αποσυζευγμένη αρχιτεκτονική (Decoupled Architecture) που υλοποιεί το πρότυπο **Client-Server**. Το σύστημα είναι χωρισμένο σε δύο ανεξάρτητα υποσυστήματα που επικοινωνούν αποκλειστικά μέσω ασφαλών, stateless κλήσεων REST API, ανταλλάσσοντας δεδομένα σε μορφή JSON.

Η σχεδίαση ακολουθεί την αρχιτεκτονική **Layered (Πολυεπίπεδη)**, διαχωρίζοντας σαφώς τις ευθύνες του κώδικα σε διακριτά επίπεδα, με σκοπό τη μεγιστοποίηση της επεκτασιμότητας, της ασφάλειας και της ευκολίας συντήρησης:

- **Επίπεδο Παρουσίασης (Presentation Layer - Frontend):** Υλοποιημένο ως Single Page Application (SPA) με Vue.js 3 και Vite. Είναι υπεύθυνο για τη διαχείριση της διεπαφής, την εγκυρότητα των δεδομένων εισόδου στην πλευρά του πελάτη (client-side validation) και τη δυναμική απόδοση των καταστάσεων του UI.
- **Επίπεδο Ελέγχου (Controller Layer - API Endpoints):** Το πρώτο επίπεδο υποδοχής στο Backend (Spring Boot). Οι `@RestController` κλάσεις αναλαμβάνουν την αποδοχή των HTTP requests, τον έλεγχο των δικαιωμάτων πρόσβασης (URL-based security) και τη χαρτογράφηση των JSON payloads σε Java Records (DTOs).
- **Επίπεδο Επιχειρησιακής Λογικής (Service Layer):** Η καρδιά του συστήματος, όπου υλοποιούνται όλοι οι επιχειρησιακοί κανόνες (Business Rules), οι ασύγχρονες ροές εργασίας (`@Async`) για την επεξεργασία των AI μοντέλων, ο υπολογισμός της βιωσιμότητας και οι αλγόριθμοι failover.
- **Επίπεδο Πρόσβασης Δεδομένων (Data Access / Persistence Layer):** Υλοποιημένο μέσω του Spring Data JPA και του Hibernate. Αναλαμβάνει τη μετατροπή των Java Entities σε σχεσιακές εγγραφές και τη διαχείριση των transactions με την PostgreSQL.

4.2 Αναλυτική Δομή Καταλόγων (Directory Structure)

Η οργάνωση του πηγαίου κώδικα έγινε με βάση τα πρότυπα της βιομηχανίας. Στο Backend εφαρμόστηκε η προσέγγιση **package-by-feature** (οργάνωση ανά επιχειρησιακή λειτουργία), η οποία επιτρέπει την πλήρη απομόνωση των ενότητων και την αποφυγή της δημιουργίας μονολιθικών πακέτων.

4.2.1 Δομή του Backend (Spring Boot)

Η αρχιτεκτονική του Backend αναπτύσσεται κάτω από τον κεντρικό κατάλογο [EstateSol/](#) και διαρθρώνεται ως εξής:

- [ai/](#): Περιλαμβάνει τη λογική διασύνδεσης με το Google Vertex AI Cloud.

- `client/`: Περιέχει την κλάση `GeminiClient.java` για τη διαχείριση των HTTP συνδέσεων και το `GoogleTokenService.java` για την ασφαλή ανάκτηση των OAuth2 tokens.
- `dto/`: immutable αντικείμενα (όπως το `PricingResult.java`) για τη μετατροπή των JSON αποκρίσεων των μοντέλων της Google.
- `service/`: Υλοποιεί το `AIProcessingService.java` (ασύγχρονη ενορχήστρωση), το `AIFeasibilityService.java` (αλγόριθμος βιωσιμότητας) και το `FallbackPricingService.java` (τοπικός μηχανισμός failover).
- `apartment/`: Το κεντρικό λειτουργικό πακέτο της εφαρμογής για τη διαχείριση των ακινήτων.
 - `controller/`: Περιλαμβάνει τον `RestApartmentController.java` (CRUD ακινήτων ιδιωτών) και τον `RestLeadPoolController.java` (διαχείριση των B2B marketplace pools και των bids).
 - `dto/`: Records μεταφοράς δεδομένων, όπως τα `ApartmentCreateDto.java`, `ApartmentUpdateDto.java` και `LeadPoolDto.java`.
 - `entity/`: Οι JPA οντότητες `Apartment.java` και `LeadBid.java`, καθώς και τα Enums των καταστάσεων `ProjectStatus.java` και `BidStatus.java`.
 - `mapper/`: Η κλάση `ApartmentMapper.java` για την ασφαλή μετατροπή των Entities σε DTOs και το αντίστροφο.
 - `repository/`: Τα interfaces `ApartmentRepository.java` και `LeadBidRepository.java` για την επικοινωνία με τη βάση.
- `auth/`: Περιέχει τον `RegisterController.java` που διαχειρίζεται αποκλειστικά τη ροή εγγραφής νέων χρηστών και το validation των εταιρικών εγγράφων.
- `payment/`: Περιλαμβάνει τον `RestPaymentController.java` και το `VivaWalletService.java` για την προσομοίωση των escrow holds και τον υπολογισμό του ελληνικού ΦΠΑ (24%).
- `profile/`: Διαχειρίζεται το κοινωνικό δίκτυο των εταιρειών. Περιέχει τα Entities, Repositories και Controllers για τα `BusinessPost` (εταιρικό wall) και τα `BusinessReview` (κριτικές συνεργασίας).
- `security/`: Περιλαμβάνει το `SecurityConfig.java` (κεντρική παραμετροποίηση του Spring Security, φίλτρα, remember-me, concurrent configuration) και το `WebConfig.java` για το static resource directory mapping.
- `storage/`: Περιέχει το `ImageStorageService.java`, το οποίο διαχειρίζεται τα binary και base64 streams για την αποθήκευση των αρχικών και των AI ανακαινισμένων εικόνων στον δίσκο του server.
- `user/`: Πακέτο διαχείρισης χρηστών και διοικητικών λειτουργιών.

-
- `controller/`: Περιέχει τον `RestAdminManagementController.java` (έγκριση εταιρειών, overrides), τον `RestAdminUserController.java` (CRUD χρηστών από τον Admin) και τον `RestMeController.java` (session context). `entity/`: Η κεντρική οντότητα `User.java` και το Enum `UserRole.java`.
- `security/`: Υλοποιήσεις του Spring Security, όπως το `CustomUserDetailsService.java` και το `CustomLoginSuccessHandler.java`.
- `service/`: Οι υπηρεσίες `UserService.java` και `AdminManagementService.java`.
- `DbConstraintCleaner.java`: Εξειδικευμένη κλάση bootstrapped migration, η οποία εκτελείται αυτόματα κατά την εκκίνηση της εφαρμογής για τον καθαρισμό παλαιών check constraints της SQL και τη διόρθωση legacy image paths.

4.2.2 Δομή του Frontend (Vue.js 3)

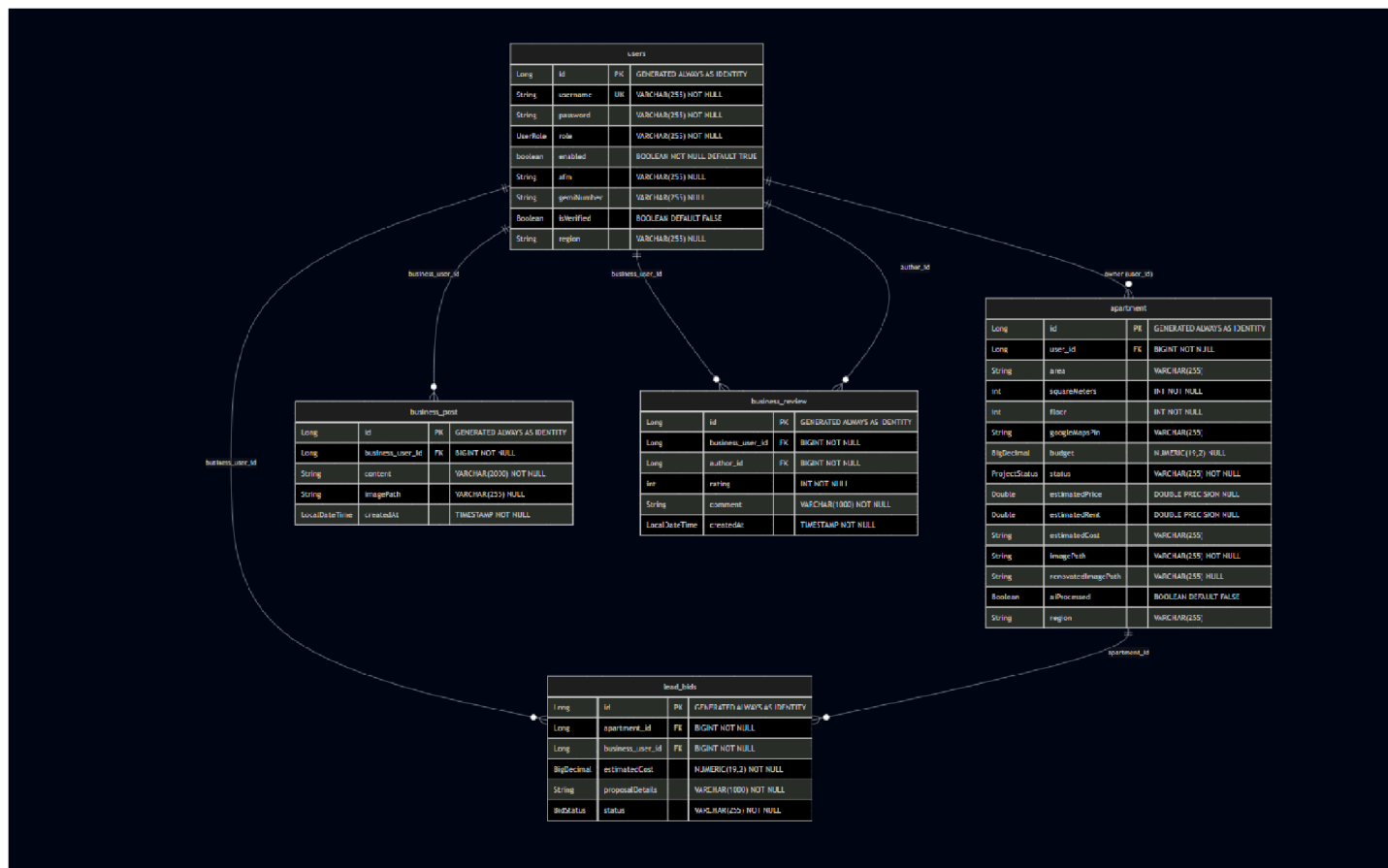
Ο κώδικας του επιπέδου παρουσίασης αναπτύσσεται στον κατάλογο `VUE-JS/` και οργανώνεται ως εξής:

- `src/main.js`: Το σημείο εισόδου και bootstrapping της Vue εφαρμογής, όπου αρχικοποιούνται τα plugins (Router, Reactive State).
- `src/App.vue`: Το ριζικό (root) component που φιλοξενεί το κεντρικό layout και το `<router-view />`.
- `src/router/index.js`: Ο κεντρικός δρομολογητής, όπου ορίζονται όλες οι διαδρομές και οι πλοηγικοί οδηγοί προστασίας (Navigation Guards).
- `src/stores/auth.js`: Το αρχείο που φιλοξενεί το παγκόσμιο custom **reactive session object** για τη διατήρηση του login state, των στοιχείων του χρήστη και της μεθόδου `checkAuth()`.
- `src/views/`: Οι κεντρικές σελίδες (Views) της πλατφόρμας, οι οποίες λειτουργούν ως smart containers:
 - `LoginView.vue` & `RegisterView.vue`: Φόρμες εισόδου και εγγραφής με ενσωματωμένο client-side validation για ΑΦΜ/ΓΕΜΗ.
 - `AdminDashboardView.vue`: Το ολοκληρωμένο περιβάλλον εργασίας του διαχειριστή με panels ελέγχου χρηστών και overrides.
 - `UserDashboardView.vue`: Δυναμικό dashboard που προσαρμόζει το περιεχόμενό του (Tabs) ανάλογα με τον ρόλο του συνδεδεμένου χρήστη (Ιδιώτης ή Εταιρεία).
 - `ApartmentDetailsView.vue`: Σελίδα αναλυτικής προβολής ενός ακινήτου με split-screen προβολή των εικόνων (πριν/μετά) και των AI τιμών.
 - `BusinessProfileView.vue`: Δημόσιο προφίλ επαγγελματία με live υπολογισμό των ratings, προβολή reviews και του post feed.
- `src/components/`: Επαναχρησιμοποιήσιμα, αυτόνομα οπτικά στοιχεία (Dumb

- Components) που επικοινωνούν με τα Views μέσω Props και Emits:
- **ApartmentCreateForm.vue**: Wizard δημιουργίας ακινήτου με drag-and-drop image upload.
- **FeasibilityWidget.vue**: Το reactive widget που εμφανίζει την έγχρωμη μπάρα βιωσιμότητας του budget.
- **ApartmentList.vue**: Renders καρτών ακινήτων με status-labeled badges.
- **ConstructionPool.vue**: Το panel αναζήτησης, φιλτραρίσματος (sliders για τ.μ., budget, περιοχή) και υποβολής bids για τις εταιρείες.
 - **BidsListForOwner.vue**: Λίστα διαχείρισης προσφορών για τους ιδιοκτήτες.
 - **UserManagement.vue**: Το admin εργαλείο διαχείρισης χρηστών και εγγράφων.

4.3 Μοντέλο Δεδομένων και Σχεσιακό Διάγραμμα (ER Diagram)

Το επίπεδο μόνιμης αποθήκευσης του EstateSol υλοποιείται σε σχεσιακό μοντέλο στην PostgreSQL, αξιοποιώντας τους κανόνες κανονικοποίησης (Normalization) για την αποφυγή πλεονασμού δεδομένων, διασφαλίζοντας παράλληλα την αναφορική ακεραιότητα (Referential



○
Integrity) μέσω ξένων κλειδιών (Foreign Keys) και μοναδικών περιορισμών (Unique Constraints).

Ακολουθεί το αναλυτικό Διάγραμμα Οντοτήτων-Συσχετίσεων (Entity-Relationship Diagram - ERD) της βάσης δεδομένων, το οποίο αποτυπώνει τη δομή των πινάκων και τις μεταξύ τους σχέσεις:

4.3.1 Ανάλυση Πινάκων και Περιορισμών Ακεραιότητας

1. Πίνακας Χρηστών (**users**):

- Φιλοξενεί όλους τους λογαριασμούς του συστήματος. Το πεδίο **username** διαθέτει περιορισμό **Unique (UK)**, εμποδίζοντας τη διπλή εγγραφή.
- Τα πεδία **afm** και **gemiNumber** είναι nullable, καθώς συμπληρώνονται υποχρεωτικά μόνο όταν το **role** έχει την τιμή **CONSTRUCTION_COMPANY** ή **REAL_ESTATE_AGENCY**.
- Συνδέεται με σχέση **One-to-Many** με τον πίνακα **apartment** (ένας ιδιοκτήτης μπορεί να έχει πολλά ακίνητα).

2. Πίνακας Ακινήτων (**apartment**):

- Το πεδίο **user_id** αποτελεί **Ξένο Κλειδί (FK)** που δείχνει στον πίνακα **users**. Έχει οριστεί με κανόνα **Cascade Type: ALL** και **Orphan Removal** σε επίπεδο JPA, πράγμα που σημαίνει ότι αν διαγραφεί ένας χρήστης από τον Admin, διαγράφονται αυτόματα και όλα τα ακίνητά του.
- Το πεδίο **status** αποθηκεύει το string representation του **ProjectStatus Enum** (**DRAFT**, **VISUALIZED**, **POOL_CONSTRUCTION**, **CONTRACT_LOCKED**, **COMPLETED**, **POOL_REAL_ESTATE**).
- Συνδέεται με σχέση **One-to-Many** με τον πίνακα **lead_bids**.

3. Πίνακας Προσφορών (**lead_bids**):

- Αποτελεί τον συνδετικό πίνακα του marketplace. Διαθέτει δύο Foreign Keys: το **apartment_id** (δείχνει στο ακίνητο του pool) και το **business_user_id** (δείχνει στην εταιρεία που υπέβαλε την προσφορά).
- Το πεδίο **estimatedCost** έχει οριστεί ως **NUMERIC(19,2)** για την αποφυγή σφαλμάτων στρογγυλοποίησης κατά τους οικονομικούς υπολογισμούς.
- Το πεδίο **status** παίρνει τις τιμές **PENDING**, **ACCEPTED**, **REJECTED**.

4. Πίνακες **business_post** και **business_review**:

- Υλοποιούν τη B2B/B2C κοινωνική δικτύωση.
- Ο πίνακας **business_review** διαθέτει μια ιδιαίτερη αρχιτεκτονική καθώς περιλαμβάνει **δύο διαφορετικά ξένα κλειδιά προς τον ίδιο πίνακα χρηστών**: το **business_user_id** (δείχνει στην εταιρεία που δέχεται την κριτική) και το **author_id** (δείχνει στον ιδιώτη χρήστη που γράφει την κριτική), διασφαλίζοντας την πλήρη ιχνηλασιμότητα των reviews.

ΚΕΦΑΛΑΙΟ 5: ΥΛΟΠΟΙΗΣΗ ΣΥΣΤΗΜΑΤΟΣ

5.1 Αναλυτική Καταγραφή και Λειτουργία των REST Controllers

Το επίπεδο ελέγχου (Controller Layer) του EstateSol υλοποιήθηκε μέσω εξειδικευμένων REST Controllers, οι οποίοι εκθέτουν προστατευμένα endpoints και διαχειρίζονται τις transactional ροές του marketplace.

5.1.1 RegisterController (/register)

Ο **RegisterController** αποτελεί την πύλη εισόδου για τη δημιουργία νέων λογαριασμών.

Διαχειρίζεται το endpoint **POST /register**, το οποίο δέχεται ως payload ένα **RegisterRequest** JSON αντικείμενο. Η λειτουργία του περιλαμβάνει:

- **Έλεγχος Μοναδικότητας:** Καλεί το **UserService** για να επαληθεύσει ότι το αιτούμενο **username** δεν υπάρχει ήδη στη βάση δεδομένων.
- **Αποκλεισμό Ανώτατων Ρόλων:** Ελέγχει και απορρίπτει αμέσως οποιαδήποτε προσπάθεια εγγραφής με ρόλο **ADMIN**, καθώς ο ρόλος αυτός δημιουργείται αποκλειστικά εσωτερικά.
- **Εταιρικό Validation (ΑΦΜ/ΓΕΜΗ):** Αν ο αιτούμενος ρόλος είναι **CONSTRUCTION_COMPANY** ή **REAL_ESTATE_AGENCY**, επιβάλλει αυστηρό έλεγχο εισόδου: το ΑΦΜ πρέπει να αποτελείται από ακριβώς 9 αριθμητικά ψηφία και να συνοδεύεται από έγκυρο αριθμό ΓΕΜΗ και γεωγραφική ζώνη δραστηριοποίησης (**region**). Ο λογαριασμός αποθηκεύεται με την ένδειξη **isVerified = false**.

5.1.2 RestMeController (/api/me)

Ο **RestMeController** εκθέτει το endpoint **GET /api/me**, το οποίο είναι προσβάσιμο από όλους τους αυθεντικοποιημένους χρήστες. Λειτουργεί ως ο κεντρικός μηχανισμός συγχρονισμού του session, επιστρέφοντας το αναγνωριστικό (**id**), το όνομα χρήστη (**username**) και τα δικαιώματα πρόσβασης (Authorities/Roles) του τρέχοντος συνδεδεμένου χρήστη από το Security Context της Spring.

5.1.3 RestPaymentController (/api/payments)

Ο **RestPaymentController** εκθέτει το endpoint **GET /api/payments/calculate-vat**. Δέχεται ως παράμετρο ένα βασικό οικονομικό ποσό (**amount**) και, καλώντας το **VivaWalletService**, εκτελεί την ανάλυση και τον υπολογισμό του επίσημου ελληνικού ΦΠΑ (συντελεστής 24%), επιστρέφοντας την ανάλυση του καθαρού ποσού, του φόρου και του τελικού συνόλου.

5.1.4 RestApartmentController (/api/apartments)

Ο **RestApartmentController** διαχειρίζεται τον κύκλο ζωής των ακινήτων:

- **GET /api/apartments/**: Επιστρέφει τη λίστα των ακινήτων. Αν ο χρήστης έχει ρόλο **ADMIN**, ανακτώνται όλα τα ακίνητα του συστήματος. Αν έχει ρόλο **AMATEUR_USER**, εφαρμόζεται αυτόματο φιλτράρισμα και επιστρέφονται αποκλειστικά τα ακίνητα των οποίων είναι ιδιοκτήτης.
- **GET /api/apartments/{id}**: Επιστρέφει τα αναλυτικά στοιχεία ενός συγκεκριμένου ακινήτου, αφού πρώτα επαληθευτούν τα δικαιώματα προβολής.
- **POST /api/apartments/createApartment**: Δέχεται multi-part form-data που περιλαμβάνει τις τεχνικές παραμέτρους (τ.μ., όροφος, budget, maps pin) σε μορφή **ApartmentCreateDto** και το binary αρχείο της αρχικής φωτογραφίας του δωματίου. Αποθηκεύει το ακίνητο με κατάσταση **DRAFT** και ενεργοποιεί το ασύγχρονο thread επεξεργασίας AI.
- **POST /api/apartments/updateasuser/{id}**: Επιτρέπει στον ιδιοκτήτη να επικαιροποιήσει τα στοιχεία του ακινήτου του. Η επιχειρησιακή λογική επιβάλλει ότι οποιαδήποτε αλλαγή στα τεχνικά χαρακτηριστικά εκτελεί αυτόματα διαγραφή όλων των υφιστάμενων προσφορών (bids), καθώς το οικονομικό αντικείμενο του έργου έχει μεταβληθεί.
- **POST /api/apartments/{id}/complete-renovation**: Μεταφέρει το status του ακινήτου σε **COMPLETED**. Είναι προσβάσιμο από τον ιδιοκτήτη, τον Admin ή την ανάδοχο τεχνική εταιρεία.
- **POST /api/apartments/{id}/reset-renovation**: Επαναφέρει το ακίνητο από οποιαδήποτε κατάσταση πίσω στο αρχικό στάδιο **DRAFT** και εκκαθαρίζει πλήρως το ιστορικό των προσφορών.

5.1.5 RestLeadPoolController (/api/pool)

Ο **RestLeadPoolController** αποτελεί την καρδιά του B2B marketplace:

- **GET /api/pool/**: Υλοποιεί τη δυναμική διανομή των leads. Αν ο συνδεδεμένος χρήστης είναι κατασκευαστική εταιρεία, επιστρέφονται μόνο τα ακίνητα με status **POOL_CONSTRUCTION**. Αν είναι κτηματομεσιτικό γραφείο, επιστρέφονται ακίνητα με status **POOL_REAL_ESTATE**.
- **POST /api/pool/{projectId}/bid**: Επιτρέπει την υποβολή προσφοράς. Το endpoint προστατεύεται με validation που ελέγχει αν η εταιρεία είναι επίσημα εγκεκριμένη (**isVerified = true**). Δέχεται το προτεινόμενο κόστος (**estimatedCost**) και το τεχνικό υπόμνημα (**proposalDetails**).
- **POST /api/pool/bids/{bidId}/accept**: Επιτρέπει στον ιδιοκτήτη να αποδεχθεί μια προσφορά. Η ενέργεια αυτή εκτελείται ως μια ατομική συναλλαγή (ACID Transaction): κλειδώνει το ακίνητο στην κατάσταση **CONTRACT_LOCKED** (escrow phase), μετατρέπει το επιλεγμένο bid σε **ACCEPTED** και αλλάζει αυτόματα το status όλων των ανταγωνιστικών bids του ίδιου pool σε **REJECTED**.

5.1.6 RestBusinessProfileController (/api/business-profile)

- **GET /api/business-profile/list**: Επιστρέφει τον δημόσιο κατάλογο των επαγγελματιών. Υπολογίζει δυναμικά και επιστρέφει για κάθε εταιρεία τον μέσο όρο των reviews της και το πλήθος των ενεργών posts της.
- **GET /api/business-profile/{userId}**: Επιστρέφει το πλήρες προφίλ μιας εταιρείας (στοιχεία, wall posts, reviews). Περιλαμβάνει τη reactive boolean μεταβλητή

`hasCooperated`, η οποία γίνεται `true` μόνο αν ο συνδεδεμένος ιδιώτης έχει ένα κλειδωμένο συμβόλαιο (`ACCEPTED` bid) με τη συγκεκριμένη εταιρεία.

- `POST /api/business-profile/posts`: Επιτρέπει σε επαληθευμένες εταιρείες να δημοσιεύουν κείμενο και φωτογραφίες από ολοκληρωμένα έργα τους στο wall τους.
- `POST /api/business-profile/{userId}/reviews`: Επιτρέπει στους ιδιώτες να υποβάλουν βαθμολογία (1 έως 10) και σχόλιο για μια εταιρεία, υπό την προϋπόθεση ότι η μεταβλητή `hasCooperated` είναι αληθής.

5.2 Core Business Services και Ασύγχρονη Ενορχήστρωση AI

5.2.1 AIProcessingService (@Async)

Η επεξεργασία των Generative AI μοντέλων της Google Cloud (Gemini και Imagen) αποτελεί μια βαριά και χρονοβόρα διαδικασία, η οποία απαιτεί από 4 έως 8 δευτερόλεπτα για να ολοκληρωθεί. Για τον λόγο αυτό, η μέθοδος επεξεργασίας στο `AIProcessingService` έχει σημειωθεί με την annotation `@Async` της Spring Boot.

Όταν ο ιδιοκτήτης καλεί το endpoint δημιουργίας ακινήτου, το `ApartmentService` αποθηκεύει άμεσα την εγγραφή στη βάση δεδομένων με κατάσταση `DRAFT` και `aiProcessed = false`, και επιστρέφει HTTP 200 OK στον client σε λιγότερο από 150ms. Το main thread ελευθερώνεται, και η AI επεξεργασία εκτελείται αυτόνομα σε ξεχωριστό background thread (νηματική απομόνωση).

5.2.2 Αλγόριθμος Feasibility (AiFeasibilityService)

Πριν γίνει οποιαδήποτε κλήση προς τα Cloud APIs, το σύστημα εκτελεί έναν τοπικό έλεγχο σκοπιμότητας μέσω του `AiFeasibilityService`. Ο αλγόριθμος υπολογίζει την αναλογία του προϋπολογισμού (Budget) ανά τετραγωνικό μέτρο του ακινήτου:

$$\text{Ratio} = \frac{\text{Budget}}{\text{Square Meters}}$$

Βάσει του αποτελέσματος, το ακίνητο κατατάσσεται σε μία από τις ακόλουθες κατηγορίες:

1. `Ratio < 200€ / τ.μ.` → `UNFEASIBLE`: Το budget κρίνεται ανεπαρκές για την εκτέλεση έστω και βασικών εργασιών. Το service διακόπτει αμέσως τη ροή, ενημερώνει τη βάση δεδομένων θέτοντας το πεδίο `estimatedCost = "UNFEASIBLE"` και `aiProcessed = true`, και ακυρώνει τις κλήσεις προς το Vertex AI, προστατεύοντας την εφαρμογή από άσκοπη σπατάλη οικονομικών πόρων.
2. `200€/τ.μ. <= Ratio < 450€/τ.μ.` → `COSMETIC_ONLY`: Ο προϋπολογισμός επαρκεί μόνο για επιφανειακές παρεμβάσεις (βάψιμο τοίχων, τοποθέτηση laminate, αλλαγή φωτιστικών).
3. `450€/τ.μ. <= Ratio < 800€/τ.μ.` → `OPTIMAL`: Ιδανική αναλογία για πλήρη εσωτερική ανακαίνιση (αντικατάσταση υδραυλικών και ηλεκτρολογικών εγκαταστάσεων, ανακατασκευή κουζίνας και μπάνιου).
4. `Ratio >= 800€ / τ.μ.` → `PREMIUM`: Επιτρέπει τη χρήση υλικών υψηλής ποιότητας, custom ξυλουργικές κατασκευές και εγκατάσταση συστημάτων smart home.
5. `Budget == null` → `BASIC_HABITABILITY`: Αν ο ιδιοκτήτης δεν ορίσει budget, το σύστημα εφαρμόζει μια βασική τιμή εκκίνησης $180\text{€} / \text{τ.μ.}$ για τον υπολογισμό των βασικών αναγκών διαβίωσης.

5.2.3 Ενοποίηση Μοντέλων Vertex AI (GeminiClient)

Αν το ακίνητο κριθεί βιώσιμο, το `GeminiClient` αναλαμβάνει τη διασύνδεση:

- **Ανάκτηση Οικονομικών Τιμών:** Μέσω του `gemini-2.5-flash`, αποστέλλεται ένα prompt που περιέχει την τοποθεσία, τα τ.μ. και την κατηγορία βιωσιμότητας. Το μοντέλο, κάνοντας χρήση Structured Output, επιστρέφει ένα αυστηρό JSON αντικείμενο με τις προβλέψεις της αγοράς. Το σύστημα διαβάζει το JSON και ενημερώνει τα πεδία `estimatedPrice` (αξία πώλησης) και `estimatedRent` (μηνιαίο ενοίκιο) του ακινήτου.
- **Παραγωγή Ανακαινισμένης Εικόνας:** Το μοντέλο `gemini-2.5-pro` αναλύει την αρχική φωτογραφία και δημιουργεί ένα prompt σχεδιασμού. Στη συνέχεια, καλείται το μοντέλο `imagen-3.0-capability-001`. Η αρχική φωτογραφία αποστέλλεται ως raw base64 reference input (`REFERENCE_TYPE_RAW`). Το Imagen 3 επεξεργάζεται το prompt σχεδιασμού και επιστρέφει τη νέα, φωτορεαλιστική εικόνα, διατηρώντας αναλλοίωτη τη γεωμετρία και τις γωνίες του χώρου. Η εικόνα αποθηκεύεται στον δίσκο μέσω του `ImageStorageService`, το status του ακινήτου μεταβάλλεται σε `VISUALIZED` και το `aiProcessed` γίνεται `true`.

5.3 Παραμετροποίηση Ασφάλειας και Διαχείρισης Συνεδριών

Η ασφάλεια του συστήματος υλοποιήθηκε στο αρχείο `SecurityConfig.java` του Spring Security, υιοθετώντας μια stateful αρχιτεκτονική βασισμένη σε Session Cookies, η οποία κρίθηκε βέλτιστη για τη διαχείριση της κατάστασης των χρηστών.

- **Απενεργοποίηση CSRF:** Η προστασία CSRF απενεργοποιήθηκε (`csrf.disable()`), καθώς η επικοινωνία με το Vue.js frontend ελέγχεται μέσω ασφαλών, custom session validations και proxying, απλοποιώντας τη δομή των REST κλήσεων.
- **URL-Based Authorization:** Ορίστηκαν αυστηρά φίλτρα πρόσβασης:
 - Τα endpoints `/login` και `/register` είναι πλήρως δημόσια.
 - Όλα τα endpoints υπό τη διαδρομή `/api/admin/` απαιτούν αποκλειστικά το δικαίωμα `ROLE_ADMIN`.
 - Τα endpoints υπό τη διαδρομή `/api/` απαιτούν αυθεντικοποιημένο χρήστη με έναν από τους ρόλους: `ROLE_AMATEUR_USER`, `ROLE_CONSTRUCTION_COMPANY` ή `ROLE_REAL_ESTATE_AGENCY`.
- **Έλεγχος Ταυτόχρονων Συνεδριών (Concurrent Session Control):** Για την αποτροπή κακόβουλης χρήσης ή διαμοιρασμού λογαριασμών, εφαρμόστηκε ο περιορισμός:

```
Java
sessionManagement(session -> session

    .maximumSessions(1)

    .maxSessionsPreventsLogin(false)

);
```

- Αν ένας χρήστης συνδεθεί από μια νέα συσκευή, το Spring Security ακυρώνει αυτόματα την παλαιότερη ενεργή συνεδρία (`invalidate`), προστατεύοντας τα transactional δεδομένα του marketplace.
- **Remember-Me Service:** Ενσωματώθηκε μηχανισμός παραμονής σύνδεσης διάρκειας 7 ημερών. Ο μηχανισμός χρησιμοποιεί το `PersistentTokenRepository` συνδεδεμένο μέσω JDBC με τον πίνακα `persistent_logins` της PostgreSQL, διασφαλίζοντας ότι τα tokens

αποθηκεύονται με ασφάλεια στη βάση δεδομένων και δεν χάνονται σε περίπτωση επανεκκίνησης του server.

5.4 Frontend Υλοποίηση και Reactive State Architecture

5.4.1 Διαχείριση Κατάστασης με Vue Reactive Session Object

Στο επίπεδο του Frontend, για τη διατήρηση της παγκόσμιας κατάστασης (Global State), χρησιμοποιήθηκε ένα custom **reactive session object** στο αρχείο [src/stores/auth.js](#), αντικαθιστώντας τα τυπικά Pinia stores:

```
JavaScript
import { reactive } from 'vue';

export const auth = reactive({
  user: null,
  isAuthenticated: false,
  role: null,
  async checkAuth() {
    try {
      const response = await fetch('/api/me');
      if (response.ok) {
        const data = await response.json();
        this.user = data;
        this.isAuthenticated = true;
        this.role = data.authority ? data.authority.replace('ROLE_', '') : null;
      } else {
        this.clear();
      }
    } catch (error) {
      this.clear();
    }
  },
  clear() {
    this.user = null;
    this.isAuthenticated = false;
    this.role = null;
  }
});
```

Η μέθοδος [checkAuth\(\)](#) καλείται αυτόματα από τους πλοηγικούς οδηγούς (**Navigation Guards**) του Vue Router πριν από τη φόρτωση κάθε προστατευμένης σελίδας. Εκτελεί ένα ασύγχρονο HTTP request στο endpoint [/api/me](#). Αν ο server επιστρέψει έγκυρο session, το reactive object ενημερώνει τις μεταβλητές του και απομονώνει τη λέξη-κλειδί του ρόλου (π.χ. μετατρέπει το [ROLE_ADMIN](#) σε [ADMIN](#)). Σε αντίθετη περίπτωση, εκκαθαρίζει το state και ανακατευθύνει τον χρήστη στη σελίδα [/login](#).

5.4.2 Σύνδεσμος Διασύνδεσης: FeasibilityWidget

Ενσωματωμένο στο component δημιουργίας ακινήτου, το `FeasibilityWidget.vue` προσφέρει μια απόλυτα reactive εμπειρία. Ακούει τις αλλαγές στα πεδία των τετραγωνικών και του budget μέσω Vue watchers. Υπολογίζει live την αναλογία και εμφανίζει μια έγχρωμη μπάρα βιωσιμότητας (Color Feasibility Bar):

- **Κόκκινο:** Για κατηγορία `UNFEASIBLE`.
- **Κίτρινο:** Για κατηγορία `COSMETIC_ONLY`.
- **Πράσινο:** Για κατηγορία `OPTIMAL`.
- **Μπλε/Μωβ:** Για κατηγορία `PREMIUM`.

Το widget εμφανίζει δυναμικά κείμενα με συμβουλές ενεργειακής αναβάθμισης και ROI, επιτρέποντας στον ιδιοκτήτη να προσαρμόσει τα οικονομικά του δεδομένα πριν πατήσει την τελική υποβολή της φόρμας.

ΚΕΦΑΛΑΙΟ 6: ΕΛΕΓΧΟΣ ΚΑΙ ΑΞΙΟΛΟΓΗΣΗ

6.1 Έλεγχος Λειτουργικότητας (Functional Testing)

Ο έλεγχος λειτουργικότητας (Functional Testing) της πλατφόρμας **EstateSol** πραγματοποιήθηκε με στόχο την επαλήθευση της ορθής εκτέλεσης των επιχειρησιακών κανόνων και των μεταβολών κατάστασης (State Transitions) των οντοτήτων στη βάση δεδομένων. Οι δοκιμές επικεντρώθηκαν στα transactional υποσυστήματα του marketplace, όπου η αλληλεπίδραση μεταξύ ιδιωτών και εταιρειών απαιτεί απόλυτη ακρίβεια.

6.1.1 Δοκιμή του Μηχανισμού Ακύρωσης Προσφορών (Bids Voiding)

Ένα από τα πιο κρίσιμα σενάρια ελέγχου αφορούσε τη συμπεριφορά του συστήματος όταν ένας ιδιώτης χρήστης τροποποιεί τα τεχνικά χαρακτηριστικά ενός ακινήτου που βρίσκεται ήδη δημοσιευμένο σε κάποιο lead pool.

- **Διαδικασία Ελέγχου:** Καταχωρήθηκε ακίνητο με αρχικό budget \$30,000\$ και μέγεθος \$80\$ τ.μ., το οποίο δημοσιεύθηκε στο `POOL_CONSTRUCTION`. Στη συνέχεια, δύο επαληθευμένες κατασκευαστικές εταιρείες υπέβαλαν προσφορές (`LeadBid`) σε κατάσταση `PENDING`. Ακολούθως, ο ιδιοκτήτης κάλεσε το endpoint `POST /api/apartments/updateasuser/{id}`, αλλάζοντας το budget σε \$20,000\$.
- **Αποτελέσματα:** Το σύστημα εκτέλεσε επιτυχώς την ενσωματωμένη επιχειρησιακή λογική του `ApartmentService`. Με την αποθήκευση των νέων στοιχείων, πυροδοτήθηκε αυτόματη cascade διαγραφή (voiding) των δύο εκκρεμών προσφορών από τον πίνακα `lead_bids`. Η δοκιμή επιβεβαίωσε την ορθότητα του κανόνα, καθώς η αλλαγή των αρχικών δεδομένων του έργου καθιστά τις προσφορές των εταιρειών παρωχημένες.

6.1.2 Δοκιμή Κλειδώματος Συμβολαίου (Contract Locking)

Ελέγχθηκε η ατομικότητα της συναλλαγής κατά την αποδοχή μιας προσφοράς από τον ιδιοκτήτη μέσω του endpoint `POST /api/pool/bids/{bidId}/accept`.

- **Αποτελέσματα:** Το σύστημα εκτέλεσε με επιτυχία μια ενιαία ACID συναλλαγή. Το status του ακινήτου μεταβλήθηκε ακαριαία από **POOL_CONSTRUCTION** σε **CONTRACT_LOCKED**, η κατάσταση του επιλεγμένου bid άλλαξε σε **ACCEPTED**, και όλες οι υπόλοιπες εκκρεμείς προσφορές για το συγκεκριμένο ακίνητο μετατράπηκαν αυτόματα σε **REJECTED**. Οι δοκιμές επιβεβαίωσαν ότι το marketplace κλειδώνει σωστά, εμποδίζοντας την υποβολή νέων προσφορών από άλλες εταιρείες κατά τη φάση του escrow hold.

6.2 Έλεγχος Ασφάλειας και Διαχείρισης Ρόλων (Security & RBAC Testing)

Η επαλήθευση της θωράκισης του συστήματος βασίστηκε σε εξαντλητικές δοκιμές ελέγχου πρόσβασης βάσει ρόλων (Role-Based Access Control - RBAC) μέσω του Spring Security, με σκοπό τη διασφάλιση των ευπαθών endpoints.

6.2.1 Δοκιμές Παραβίασης Δικαιωμάτων (Unauthorized Access Testing)

Πραγματοποιήθηκε προσομοίωση επιθέσεων με τη χρήση εργαλείων δοκιμής APIs (Postman), όπου εστάλησαν HTTP requests σε endpoints που προστατεύονται από method-level security:

- **Δοκιμή 1:** Χρήστης αυθεντικοποιημένος με ρόλο **AMATEUR_USER** επιχείρησε να καλέσει το endpoint **POST /api/admin/management/verify/{userId}** για να εγκρίνει μια εταιρεία. Το Spring Security εντόπισε την έλλειψη του **ROLE_ADMIN**, μπλόκαρε την εκτέλεση της μεθόδου πριν φτάσει στο Service επίπεδο και επέστρεψε HTTP Status Code **403 Forbidden**.
- **Δοκιμή 2:** Μη επαληθευμένη κατασκευαστική εταιρεία (**isVerified = false**) επιχείρησε να υποβάλει προσφορά στο endpoint **POST /api/pool/{projectId}/bid**. Η επιχειρησιακή λογική του Controller απέρριψε το αίτημα, επιστρέφοντας σφάλμα προστασίας, επιβεβαιώνοντας ότι μόνο εγκεκριμένοι επαγγελματίες μπορούν να συμμετέχουν στο marketplace.

6.2.2 Δοκιμή Concurrency και Remember-Me

- **Έλεγχος Session:** Πραγματοποιήθηκε προσπάθεια ταυτόχρονης σύνδεσης στον ίδιο λογαριασμό από δύο διαφορετικούς browsers. Η παραμετροποίηση **.maximumSessions(1)** λειτούργησε ορθά, ακυρώνοντας αμέσως το session cookie του πρώτου browser (session invalidation) μόλις ολοκληρώθηκε το login στον δεύτερο.
- **Έλεγχος Remember-Me:** Επιβεβαιώθηκε η δημιουργία της εγγραφής στον πίνακα **persistent_logins** της PostgreSQL κατά το κλείσιμο του browser, επιτρέποντας την αυτόματη επανείσοδο του χρήστη χωρίς τη χειροκίνητη πληκτρολόγηση κωδικών.

6.3 Αξιολόγηση AI Συμπεριφοράς και Μηχανισμού Fallback

Η αξιολόγηση του ασύγχρονου background thread (**@Async**) και της σταθερότητας της AI μηχανής αποτέλεσε έναν από τους πιο κρίσιμους δείκτες για την ποιότητα του λογισμικού.

6.3.1 Δοκιμή Συμπεριφοράς Rate Limiting (HTTP 429 Failover)

Κατά τη φάση της αξιολόγησης, πραγματοποιήθηκε τεχνητός κορεσμός των κλήσεων προς το Google Vertex AI API, προκειμένου να εξαναγκαστεί ο server σε σφάλμα υπέρβασης ορίων κλήσεων (Rate Limiting / Quota Exceeded).

- **Ενεργοποίηση Failover:** Μόλις η **WebClient** της Spring WebFlux έλαβε HTTP Status Code 429 από τα Cloud endpoints της Google, η κλάση **AIProcessingService** έκανε catch το αντίστοιχο **WebClientResponseException**. Αντί να διακόψει τη λειτουργία της εφαρμογής ή να αφήσει το ακίνητο σε κολλημένη κατάσταση επεξεργασίας, ενεργοποίησε άμεσα το **FallbackPricingService**.
- **Απόδοση Τοπικών Εκτιμήσεων:** Το Fallback Service ανέκτησε τις στατικές τιμές από τους εσωτερικούς πίνακες τιμών (pricing charts) για τη συγκεκριμένη ελληνική περιοχή (π.χ. Παγκράτι). Οι τιμές πώλησης και ενοικίου υπολογίστηκαν τοπικά, το πεδίο **estimatedCost** ενημερώθηκε με τη σχετική ένδειξη, και το entity αποθηκεύτηκε επιτυχώς με **aiProcessed = true**. Η δοκιμή απέδειξε ότι η πλατφόρμα διαθέτει υψηλή ανθεκτικότητα (Fault Tolerance) και παραμένει 100% λειτουργική ακόμα και σε συνθήκες πλήρους αποσύνδεσης από το Cloud της Google.

ΚΕΦΑΛΑΙΟ 7: ΣΥΜΠΕΡΑΣΜΑΤΑ & ΜΕΛΛΟΝΤΙΚΕΣ ΕΠΕΚΤΑΣΕΙΣ

7.1 Συμπεράσματα

Η ολοκλήρωση του σχεδιασμού, της ανάπτυξης και του ελέγχου της πλατφόρμας **EstateSol** απέδειξε ότι ο συνδυασμός enterprise αρχιτεκτονικών backend με σύγχρονα reactive frontend frameworks και cloud τεχνολογίες Παραγωγικής Τεχνητής Νοημοσύνης μπορεί να επιφέρει ριζικές, θετικές αλλαγές σε παραδοσιακά αδιαφανείς κλάδους της οικονομίας.

Η επιλογή της Java 17 και του Spring Boot 3.5.9 για το Backend εξασφάλισε ένα εξαιρετικά σταθερό, ασφαλές και πλήρως «έτοιμο για παραγωγική λειτουργία» (Deployment-Ready) πληροφοριακό σύστημα, ικανό να διαχειρίζεται πολύπλοκες transactional ροές και ασύγχρονα background threads με απόλυτη αξιοπιστία. Παράλληλα, η Vue.js 3, μέσω του Composition API και του ελαφρού, reactive session management στο **auth.js**, προσέφερε μια ταχύτατη διεπαφή χρήστη, ελαχιστοποιώντας τους χρόνους φόρτωσης και εξαλείφοντας το μπλοκάρισμα του UI κατά τη διάρκεια των απαιτητικών κλήσεων Generative AI.

Από επιχειρηματική και κοινωνική σκοπιά, το EstateSol επιτυγχάνει τον πρωταρχικό του στόχο:

- Εκδημοκρατίζει την πληροφορία γύρω από το κόστος και την απόδοση των ανακαινίσεων στην Ελλάδα, προστατεύοντας τον ιδιώτη από λανθασμένες επενδυτικές επιλογές ή υποκειμενικές κοστολογήσεις.
- Παρέχει στις ελληνικές τεχνικές και μεσιτικές εταιρείες ένα στοχευμένο ψηφιακό εργαλείο εύρεσης επαληθευμένων πελατών (Lead Generation), μειώνοντας δραστικά τα λειτουργικά τους έξοδα και διασφαλίζοντας τον υγιή ανταγωνισμό μέσω της αυστηρής διαδικασίας έγκρισης ΑΦΜ και ΓΕΜΗ.

7.2 Προβλήματα και Προκλήσεις κατά την Ανάπτυξη

Κατά τη διάρκεια του κύκλου ζωής ανάπτυξης του λογισμικού αντιμετωπίστηκαν σημαντικές τεχνικές προκλήσεις, οι οποίες απαιτούσαν εξειδικευμένες αρχιτεκτονικές λύσεις:

1. **Διαχείριση Περιορισμών SQL και Legacy Δεδομένων:** Μια σημαντική πρόκληση κατά τη φάση των updates του κώδικα ήταν η ύπαρξη παλαιών check constraints της SQL (όπως τα `users_role_check` και `apartment_status_check`) που είχαν δημιουργηθεί αυτόματα από παλαιότερες εκδόσεις του Hibernate, εμποδίζοντας τη μετάβαση στα νέα Enums. Το πρόβλημα επιλύθηκε οριστικά με την υλοποίηση της κλάσης `DbConstraintCleaner.java`. Η κλάση αυτή εκτελείται αυτόματα κατά το bootstrapping της εφαρμογής, εκτελεί raw SQL ALTER commands για το drop των παλαιών constraints, διορθώνει stuck flags επεξεργασίας και χαρτογραφεί legacy image paths σε έγκυρα αρχεία στον δίσκο, διασφαλίζοντας την ομαλή εκκίνηση του συστήματος.
2. **Συγχρονισμός Ασύγχρονων Νημάτων και Transactions:** Η διαχείριση της annotation `@Async` για το `AIProcessingService` εμφάνισε προκλήσεις συγχρονισμού (race conditions) με το persistence layer του Hibernate. Επειδή η AI επεξεργασία εκτελείται σε απομονωμένο thread, η ενημέρωση της οντότητας `Apartment` μετά την ολοκλήρωση των κλήσεων Vertex AI έπρεπε να γίνει με προσεκτική απομόνωση των transactions, ώστε να μην δημιουργούνται transient σφάλματα ή κλειδώματα (deadlocks) στη βάση δεδομένων PostgreSQL.
3. **Διαμόρφωση CORS και Διαχείριση Session Cookies:** Ο σωστός διαμοιρασμός του session cookie (`JSESSIONID`) της Spring Security μεταξύ του dev server του Vite (Frontend - Port 5173) και του Tomcat (Backend - Port 8080) επιλύθηκε μέσω της παραμετροποίησης ειδικού client-side API proxying στο αρχείο `vite.config.js`, εξαλείφοντας τα σφάλματα Cross-Origin Resource Sharing (CORS).

7.3 Μελλοντικές Επεκτάσεις (Future Work)

Η πλατφόρμα EstateSol έχει σχεδιαστεί με βάση τις αρχές της επεκτασιμότητας, γεγονός που επιτρέπει την εύκολη ενσωμάτωση νέων λειτουργιών σε μελλοντικές εκδόσεις:

- **Live Ενοποίηση με Viva Wallet Smart Checkout:** Αν και αυτή τη στιγμή εφαρμόζονται οι υπολογιστικές προδιαγραφές (όπως ο υπολογισμός του ελληνικού ΦΠΑ 24% μέσω του `RestPaymentController`), μελλοντική επέκταση αποτελεί η πλήρης, live διασύνδεση με το API της Viva Wallet. Αυτό θα επιτρέψει την πραγματική εκτέλεση των simulated escrow μεθόδων `holdFunds` και `releaseMilestoneFunds`, δεσμεύοντας την προκαταβολή κατά την αποδοχή της προσφοράς (`ACCEPTED` bid) και αποδεσμεύοντάς την τμηματικά στον επαγγελματία με βάση τα κατασκευαστικά ορόσημα (milestones).
- **Αυτοματοποιημένη Live Διασταύρωση με το API του ΓΕΜΗ:** Για την πλήρη εξάλειψη της ανάγκης χειροκίνητου ελέγχου των εταιρικών στοιχείων από τον Admin, προτείνεται η ενσωμάτωση web services ή απευθείας διασύνδεσης με το επίσημο API του Γενικού Εμπορικού Μητρώου. Έτσι, κατά την υποβολή της φόρμας εγγραφής, το σύστημα θα επαληθεύει ακαριαία αν το ΑΦΜ και το ΓΕΜΗ αντιστοιχούν σε ενεργή τεχνική ή μεσιτική εταιρεία, ενεργοποιώντας αυτόματα τον λογαριασμό (`isVerified = true`).
- **Ανάπτυξη Native Mobile Εφαρμογής:** Η reactive αρχιτεκτονική της Vue.js 3 επιτρέπει την εύκολη μετατροπή του υφιστάμενου frontend κώδικα σε υβριδική mobile εφαρμογή (iOS & Android) με τη χρήση του framework `Capacitor.js`. Η επέκταση αυτή θα επιτρέψει στους ιδιοκτήτες ακινήτων να χρησιμοποιούν τη native κάμερα της συσκευής τους για τη λήψη φωτογραφιών των δωματίων και την άμεση μεταφόρτωσή τους στο EstateSol, βελτιώνοντας ακόμα περισσότερο την ευχρηστία της πλατφόρμας.